BARGE_for_review

> **Paper**: *Bridging the Structural Gap: Adapting Autoregressive Generation for Recommendation*
>
> This document is the supplementary material accompanying our submission. Due to licensing constraints the source code cannot be released at this time; instead, this file provides (i) detailed responses to reviewer comments; (ii) additional experiments; (iii) the description of every module introduced by BARGE, at a level of detail sufficient for a competent practitioner to re-implement the method from scratch.

Contents

Part I. Detailed Responses to Reviewer Comments

> The following sections provide point-by-point responses to each reviewer. We have organized the replies by reviewer and by individual question for quick access. We sincerely thank all reviewers for your careful reading and insightful comments!

Reviewer 1

W1. Novelty over related methods

> The novelty over related methods is not sufficiently established. ICA resembles item-level pooling and gated fusion, HPR resembles contrastive beam reranking, and DPD resembles multi-view ensemble decoding. The differences from TrieRec, PROMISE, APAO, and other generative recommenders should be compared more formally.

Reply:

The distinction can be summarized along three axes:

Method	Item-level structure	Semantic-drift mitigation	Multi-view item decoding
TrieRec	Prefix-tree-aware bias	×	×
Promise	×	Process reward model	×
APAO	×	Prefix-aware loss	×
BARGE	**Item-level content re-injection**	**Semantic-consistency reranking**	**Multi-codebook multi-view decoding**

BARGE is the only work that jointly targets all three axes—item-level structure, drift mitigation, and multi-view decoding—while prior methods each address only one.

- **TrieRec** introduces topology-aware relative-position bias based on the common-prefix depth between SIDs. BARGE instead explicitly constructs item-level context and adaptively injects it into the encoder representation.
- **Promise** applies a process reward model to rerank generated candidates. BARGE uses a lightweight dual-tower HPR to perform semantic-consistency verification along the hierarchical generation path.
- **APAO** emphasizes token-level loss weighting. BARGE complements this line of work with multi-view generation, where complementary code streams are decoded independently and their candidate item sets are subsequently fused.

Thus the contribution is not simply an alternative reranker or another decoder. BARGE provides a coordinated design covering **item-level structure, semantic drift, and multi-view item generation**. The three components are complementary: ICA acts on encoder-side explicit item inductive bias, DPD changes the representation and decoding paths, and HPR constrains the generated paths at inference time. We will elaborate these distinctions in the revision.

W2. Isolation of improvement sources (DPD capacity)

> The experiments do not fully isolate the sources of improvement. In particular, DPD uses two decoders and two beam searches, so its gains may partly result from additional decoding capacity rather than orthogonal representations.

Reply:

As analyzed in §II.A.2, the gain does **not** stem from the extra decoder alone.

We directly test this concern on Amazon Beauty and Sports. The key control is a **module-free TIGER reproduction** using the same codebook, compared with a two-decoder ensemble that does not use the full BARGE design.

Amazon Beauty

Variant	R@5	N@5	R@10	N@10
Module-free (top20)	0.0450	0.0309	0.0696	0.0389
Module-free (top40)	0.0473	0.0317	0.0735	0.0402
DPD → Ensemble	0.0601	0.0412	0.0879	0.0502
BARGE	**0.0654 ± 0.0006**	**0.0460 ± 0.0015**	**0.0927 ± 0.0004**	**0.0547 ± 0.0007**

Amazon Sports

Variant	R@5	N@5	R@10	N@10
Module-free (top20)	0.0257	0.0168	0.0405	0.0216
Module-free (top40)	0.0259	0.0169	0.0420	0.0221
DPD → Ensemble	0.0298	0.0199	0.0452	0.0248
BARGE	**0.0369 ± 0.0009**	**0.0252 ± 0.0016**	**0.0544 ± 0.0007**	**0.0308 ± 0.0008**

Module-free denotes a faithful TIGER reproduction with the same codebook and no added BARGE modules. **DPD → Ensemble** replaces OSQ-VAE by the original RQ-VAE and trains two decoders independently, isolating the effect of simply having multiple decoders.

The gap between DPD → Ensemble and BARGE shows that the improvement cannot be attributed to decoder multiplicity alone. The additional gain comes from the combination of the complementary OSQ-VAE representation, ICA, HPR, and the corresponding multi-view decoding/fusion design.

We additionally replace the learned orthogonal transformation in OSQ-VAE by a fixed random matrix. This control further examines whether the observed gain is simply caused by injecting an arbitrary transformation:

Variant	Beauty R@5	Beauty N@5	Beauty R@10	Beauty N@10
Fixed random matrix	0.0591	0.0420	0.0845	0.0502
BARGE	**0.0654 ± 0.0006**	**0.0460 ± 0.0015**	**0.0927 ± 0.0004**	**0.0547 ± 0.0007**

Variant	Sports R@5	Sports N@5	Sports R@10	Sports N@10
Fixed random matrix	0.0309	0.0213	0.0472	0.0265
BARGE	**0.0369 ± 0.0009**	**0.0252 ± 0.0016**	**0.0544 ± 0.0007**	**0.0308 ± 0.0008**

Together, these controls indicate that the gain is attributable to the overall structural design rather than merely to adding another decoder or an arbitrary projection.

W3. Standard deviations and industrial baselines

> The Tencent offline experiment compares only with TIGER, and no standard deviations or significance tests are reported for the public datasets.

Reply:

Results are now supplemented with standard deviations on the public datasets (see the BARGE rows in §II.A.1 and §II.A.2). For the industrial offline comparison we additionally report statistical

significance (*: $p < 0.05$) against the strongest production baseline (OneRec); see §II.A.4.

The industrial offline table now includes three strong baselines:

Method	H@5	H@10	H@20	H@50
GraphSAGE-based [2]	0.2932	0.3743	0.4650	0.5951
NANN [3]	0.4416	0.4946	0.5636	0.6760
OneRec	0.5459	0.6132	0.6729	0.7348
BARGE	**0.6015***	**0.6510***	**0.6967***	**0.7520***

* : $p < 0.05$. Data scale: >100M users, >300k items, >100M samples.

D1. Significance tests and online A/B details

> Please report standard deviations and statistical significance. For the online test, provide confidence intervals or p-values, experiment duration, and sample sizes.

Reply:

Standard deviations are reported for all public-dataset BARGE results (see §II.A.1 and the controlled DPD tables in §II.A.2). Industrial offline results include $p < 0.05$ markers.

For the online A/B test (§II.A.5):

Metric	p-value	Significant?
Exposure conversion rate	1.08×10^{-5}	✓ ($p < 0.05$)
Platform total watch duration	0.04	✓ ($p < 0.05$)

Statistical procedure: two-group hypothesis testing; T-test for regular metrics, Mann–Whitney U (rank-sum) for long-tail/skewed metrics; CUPED for variance reduction; BH-FDR correction with $q \leq 0.05$. Full sample sizes and experiment duration are internal to the production platform and cannot be disclosed under company policy; the reported p-values already incorporate the production traffic volume.

D2. Pairwise module combinations

> It would be useful to report pairwise module combinations, e.g., ICA+HPR, ICA+DPD, and HPR+DPD, to support the claim that the modules are complementary.

Reply:

Leave-one-module-out ablations (already present in the main paper and expanded here) show that removing any single component degrades performance. On Amazon Beauty:

Variant	R@5	N@5	R@10	N@10
BARGE w/o ICA	0.0611	0.0434	0.0868	0.0516
BARGE w/o HPR	0.0612	0.0437	0.0866	0.0519
BARGE w/o DPD	0.0559	0.0391	0.0768	0.0464
BARGE	**0.0654 ± 0.0006**	**0.0460 ± 0.0015**	**0.0927 ± 0.0004**	**0.0547 ± 0.0007**

On Amazon Sports:

Variant	R@5	N@5	R@10	N@10
BARGE w/o ICA	0.0339	0.0232	0.0497	0.0283
BARGE w/o HPR	0.0344	0.0228	0.0515	0.0284
BARGE w/o DPD	0.0308	0.0207	0.0469	0.0259
BARGE	**0.0369 ± 0.0009**	**0.0252 ± 0.0016**	**0.0544 ± 0.0007**	**0.0308 ± 0.0008**

DPD contributes the largest individual gain; ICA and HPR each provide consistent additional improvements. Pairwise combinations are implicitly covered by the leave-one-out design. Full pairwise tables and further ablations appear in the anonymous supplementary material linked in the rebuttal.

D3. Inference configuration and efficiency

> The hardware is specified as two NVIDIA H20 GPUs, but the efficiency experiment still needs the inference batch size, beam width, numerical precision, timing procedure, and GPU allocation. The comparison should also use the same encoder depth for TIGER and BARGE.

Reply:

All compared models use the same inference protocol: batch size 256, beam width 20, BF16 precision. GPU utilization stays near 70% and per-GPU memory near 4 GB under this configuration (batch size can be adjusted according to available GPU capacity).

A detailed latency breakdown is given in §II.A.6:

Module	Share of total inference time
Encoder (including ICA)	**1.84%**
DPD (per-view decoding + prefix filtering + view-to-item mapping)	**94.63%**
HPR (path embedding + dual-tower scoring + fusion)	**0.98%**
Other	≈ 2.55%

HPR contributes only ≈1% of total inference time because each candidate requires only small projection operations and a similarity computation; scores are produced in a single batched forward pass. DPD consists of two standard autoregressive beam searches over the shared encoder memory and does not introduce an extra serial stage. Its primary resource trade-off is increased GPU memory/utilization for the second decoder, KV-cache and beam tensors.

D4. Industrial baselines and dual-channel SIDs

> Please include more baselines on the Tencent dataset and clarify how the two channel-specific semantic IDs are combined at the shared encoder input. This information is important for evaluating the method and reproducing it.

Reply:

More industrial baselines (GraphSAGE-based, NANN, OneRec) are now reported in §II.A.4 together with statistical significance. Online A/B results appear in §II.A.5.

Dual-channel SID representation. Each view maintains its own SID embedding table. For an encoder input token that corresponds to a particular layer and view, the model retrieves the corresponding embeddings from the two view-specific tables, concatenates them, and applies an MLP to map the concatenated representation back to the required hidden dimension. This design lets the

two views keep separate discrete semantics while still supplying the shared encoder with a unified continuous representation.

Reviewer 2

W1. ICDE / data-engineering relevance

> The paper reads primarily as a new neural architecture for recommender systems. It contains little data-engineering contribution beyond a small timing comparison. A paper which purely advances machine-learning or data-mining methods without a concrete data-engineering connection may not be so relevant to ICDE.

Reply:

We believe the contribution is not limited to neural architecture design. From a data-engineering perspective, BARGE introduces two important ways of organizing and representing the item space for generative retrieval.

First, the decreasing codebook allocation across RQ layers creates a hierarchical semantic-ID space, which can be viewed as a data-dependent hierarchical indexing structure: early levels provide a larger and coarser partition, while later levels refine the representation.

Second, OSQ-VAE decomposes an item embedding into complementary semantic subspaces and assigns each subspace its own code stream. This is analogous to a multi-way indexing strategy, where the same item is represented through multiple complementary views rather than a single flat semantic-ID path.

The core question addressed by BARGE is therefore not merely “how to make a new architecture”, but **how to organize and represent item data so that autoregressive generation can retrieve items more faithfully**. This is the data-centric aspect of the method and aligns with ICDE’s focus. Moreover, we note that many outstanding papers in the recommendation community have been published at ICDE in previous years. We hope to follow in the footsteps of these works and contribute to ICDE.

W2. DPD vs. ordinary ensemble

> DPD is not clearly distinguished from an ordinary ensemble. DPD uses two separately parameterized decoders, two channel-private HPR modules, two semantic-ID spaces, and an

> independent beam search of width B in each channel. Thus, DPD receives approximately twice the decoding branches and search resources of a single-channel method. Truncating the final union to the same K does not make the computational or search budget equal. The statement that OR-fusion "does not enlarge the candidate budget" is therefore misleading.

Reply:

Sorry for the earlier ambiguity. The controlled experiments that isolate the contribution of the extra decoder are given in §II.A.2 (also summarized under Reviewer 1 W2).

In short: replacing OSQ-VAE by the original RQ-VAE while retaining two independently trained decoders ("DPD → Ensemble") improves over the module-free baseline, yet remains consistently below full BARGE. Increasing the beam width of a single decoder from 20 to 40 also fails to close the gap. Thus the observed gain cannot be explained by decoder multiplicity or search-budget enlargement alone; it arises from the joint design of the complementary multi-view representation, ICA, HPR and the final OR-fusion.

W3. Item-boundary gap / ICA motivation

> The claimed "item-boundary gap" might be flawed. Since each item has a fixed-length L-token ID, its boundaries can be recovered from token positions and codebook levels. ICA therefore may add a conventional item-level pooling bias rather than restoring genuinely lost structure. Therefore, the paper's core motivation and novelty claim might be flawed.

Reply:

We fully agree with the important distinction raised in your question!

Item boundaries do not disappear positionally. Because each item has a fixed-length L-token representation, the encoder can in principle infer where item boundaries occur from token positions. Therefore, it would be imprecise to claim that ICA "recovers" boundaries that have become irreversibly unavailable.

The actual issue is different: although boundaries are *recoverable from positions*, a standard Transformer encoder is not explicitly constrained to perform **item-level aggregation** over those groups of L tokens. The sub-tokens of one item participate in the same global self-attention stream, and nothing in the standard architecture requires the representation of an item to be explicitly summarized before being propagated through the sequence.

We therefore interpret ICA as an **explicit item-level inductive bias**, not as a boundary-recovery mechanism.

Concretely, ICA first groups the encoder input into (B, T, L, D), performs a one-query cross-attention over the L sub-tokens belonging to each item, obtains one item-level context vector, projects it, and broadcasts it back to that item's sub-token positions through a learned sigmoid gate. The operation is therefore explicitly tied to item granularity.

Is ICA merely an ordinary pooling operation?

To address this concern directly, we compare ICA with a simple mean-pooling replacement under the same downstream recommendation setting (§II.A.9):

Variant	R@5	N@5	R@10	N@10
Mean-pooling	0.0599	0.0434	0.0838	0.0512
BARGE (ICA)	**0.0622**	**0.0439**	**0.0886**	**0.0524**

ICA improves over mean pooling on all four metrics. The difference is architectural rather than merely the existence of an item-level summary: mean pooling applies a fixed aggregation rule, while ICA learns a query-dependent attention distribution and a feature-dependent gate. We sincerely appreciate the your incisive observation; the revised manuscript will adopt the clearer terminology above.

W4. Top-2k vs. top-k formulation

> There is an applicability mismatch between the DPD OR-fusion analysis and the stated evaluation protocol. Equation (19) applies to the union of two per-channel top-K sets, whose size can be as large as 2K. It does not generally apply after the union is rescored and truncated back to K. Thus the statement that OR-fusion misses iff both channels miss is not justified under the fixed-K protocol.

Reply:

You are absolutely right! Equation (19) is well-defined only within the expanded top-$2k$ candidate set; after scoring, the candidate list is truncated to top-k.

In the implementation the procedure is therefore:

1. construct the expanded candidate set;
2. compute the reranking/fused score over that set;

3. retain the top-k candidates according to the fused score.

We will make this ordering explicit in the revised description to avoid suggesting that the expression is evaluated over an already truncated candidate set.

W5. Module isolation and equal-compute controls

> The contribution of individual modules is not isolated. Table V lacks a module-free control with the same backbone and codebook, leave-one-module-out variants, and pairwise combinations. Moreover, DPD employs two decoders, each with its own beam search, so its gains may arise from increased search capacity or generic ensembling rather than the proposed orthogonal semantic decomposition. Equal-compute controls, such as a conventional two-decoder ensemble or a single decoder with a correspondingly larger beam, are needed to validate the claimed mechanism.

Reply:

Addressed in §II.A.2 and under Reviewer 1 W2 / D2. The module-free top-20 / top-40 controls, the DPD → Ensemble control, the fixed-random-matrix control, and the leave-one-module-out tables collectively isolate the contribution of decoder multiplicity from the contribution of the complete BARGE design.

W6. Limited public evaluation

> The public evaluation is limited to two small, closely related Amazon datasets, and the private offline test compares only against TIGER.

Reply:

Please refer to §II.A.1 for a full, independently reproduced Amazon Toys comparison against SASRec, TIGER, HSTU, ActionPiece, APAO-pointwise, Promise and RPG. Industrial results with multiple strong baselines appear in §II.A.4.

D1. Orthogonality of HPR and DPD

> The paper repeatedly claims that the semantic drift is addressed from two "orthogonal" angles using HPR and DPD. However, empirical results do not support such claim of orthogonality. For example, using TIGER as the reference, the Beauty R@10 improvements are $\delta_H PR$ = 0.0864−0.0648=0.0216, and $\delta_D PD$ = 0.0913−0.0648=0.0265. If the gains were approximately additive, one might expect a value near 0.0648+0.0216+0.0265=0.1129. The full model instead obtains 0.0927.

Reply:

By “orthogonal” we mean **different design angles**, not statistically independent effects.

- HPR performs **intra-path semantic consistency verification** during hierarchical decoding.
- DPD performs **multi-view item prediction**, allowing complementary SID streams to generate candidates from different representational views.

Because both modules act at inference time, their successful candidates can overlap (e.g., an item rescued by HPR may already have been generated by the other decoder). Consequently we do not claim that the gains are strictly additive; the term “orthogonal” refers only to their distinct mechanistic roles. We apologize for the misunderstanding, and it was indeed our description error.

D2. Claim / table inconsistencies

> There are some claim/table inconsistencies. On Beauty R@10, APAO-pointwise (0.0795), not ActionPiece (0.0775), is the strongest baseline, so the claimed +19.6% gain is about +16.6% against the actual strongest baseline. In Table V, RRF has higher Beauty R@10 than LSE (0.0931 vs. 0.0927), contradicting the statement that LSE is best on every metric. Fig. 4's caption also says that all four configurations are fully learned and use no random collision ID, whereas Section V-H defines the three-layer TIGER configuration as including a random collision-resolving ID.

Reply:

We did overlook updating the text after adding APAO. LSE does under-perform RRF on one metric, and the Fig. 4 caption was also incorrect. All writing issues have now been fixed: the relative-gain claim is stated against the true strongest baseline, the LSE/RRF comparison is qualified, and the figure caption has been corrected. Many thanks for catching these inconsistencies!

Reviewer 3

W1. Baseline reproducibility on Toys

> Public benchmarks are relatively small, and many baseline numbers are taken from prior work, which raises reproducibility concerns.

Reply:

We originally aimed to reuse reported metrics as in TIGER / COBRA / ActionPiece. To fully address the concern, on Amazon Toys we have now reproduced all baselines ourselves under the same evaluation protocol. Results are given in §II.A.1:

Method	R@5	N@5	R@10	N@10
SASRec	0.0474	0.0321	0.0666	0.0382
TIGER	0.0499	0.0325	0.0769	0.0412
HSTU	0.0532	0.0390	0.0713	0.0448
ActionPiece	0.0467	0.0306	0.0735	0.0392
APAO-pointwise	0.0512	0.0339	0.0772	0.0423
Promise	0.0493	0.0321	0.0746	0.0402
RPG [1]	0.0579	0.0388	0.0863	0.0479
BARGE	**0.0622 ± 0.0018**	**0.0439 ± 0.0019**	**0.0886 ± 0.0012**	**0.0524 ± 0.0012**

[1] Hou et al., “Generating long semantic ids in parallel for recommendation,” KDD 2025.

W2. Efficiency analysis

> Efficiency analysis is limited; HPR Top-N expansion and dual decoders may increase inference cost, but large-scale latency and memory are not fully reported.

Reply:

Please see §II.A.2 and the latency breakdown in §II.A.6. ICA and HPR together account for only a few percent of end-to-end inference time; HPR scores are produced in a single batched forward pass, so changing the top-N shortlist size has negligible impact on latency. DPD is the dominant cost because

it contains the autoregressive beam searches; the two views run side-by-side over the shared encoder memory and do not introduce a new serial stage. The primary extra resource is GPU memory for the second decoder, KV-cache and beam tensors.

W3. Two-stage training / codebook design

> The frozen OSQ-VAE two-stage training may limit end-to-end optimality, and the codebook design contributes nontrivially to the gains.

Reply:

Recent work has begun exploring end-to-end optimization of generative recommendation, but industrial deployability of fully end-to-end alternatives remains an open question. We therefore build on the established autoregressive paradigm.

Importantly, the proposed design is not tied specifically to RQ-VAE. DPD requires complementary code views and independent decoding paths; the same design can be applied on top of other semantic-ID construction mechanisms, including RQ-KMeans-style tokenizers. This is part of the intended lightweight and transferable nature of the method. The extra gains that come from the orthogonal multi-view representation are a core, intentional part of DPD rather than an unintended side-effect of the codebook.

W4. Online A/B details

> Online A/B details are sparse, including baseline system, duration, statistical testing, and traffic segmentation.

Reply:

Also addressed under Reviewer 1 D1 and in §II.A.5. The online experiment compares BARGE against the incumbent production baseline on the industrial platform (>100 M users, >300 k items). Both core metrics (exposure conversion rate and platform total watch duration) pass the $p < 0.05$ threshold after CUPED variance reduction and BH-FDR correction. Exact experiment duration and absolute traffic volume are internal and cannot be disclosed; the reported p-values already reflect the production sample sizes.

D1. Full inference cost

> Provide full inference cost under the same beam width, including HPR scoring and dual-channel decoding.

Reply:

The detailed latency breakdown under identical beam width, batch size and precision is given in §II.A.6 and summarized under Reviewer 1 D3. HPR accounts for ≈ 0.98 % of total inference time; DPD (the two parallel beam searches) accounts for ≈ 94.6 %.

D2. More baselines under identical budgets

> Compare against more recent generative retrieval and ANN-augmented baselines under identical candidate budgets.

Reply:

New baselines have been added on Amazon Toys (SASRec, HSTU, ActionPiece, APAO-pointwise, Promise, RPG) under the same evaluation setting; see §II.A.1. On the industrial catalog the comparison includes GraphSAGE-based, NANN and OneRec (§II.A.4).

D3. Code and evaluation protocols

> Release code, prompts, tokenizer training scripts, and evaluation protocols.

Reply:

Due to company policy, full open-sourcing may follow later. For the review period we provide an anonymous repository that records implementation details and the extra experiments that could not fit into the main reply:

https://anonymous.4open.science/r/BARGE_for_review-4840

Tokenizer training and evaluation protocols follow TIGER. The raw item input concatenates Title, Brand, Categories, Price, salesRank and Description. The complete module-level specification (tensor shapes, ICA placement, dual-decoder architecture, HPR training objective, beam-search fusion, two-stage pipeline) is given in Part III of this document.

D4. Cold-start, collisions, scaling, domain shift

> Analyze cold-start items, codebook collisions, catalog size scaling, and domain shift.

Reply:

All four analyses are provided in the common evidence sections:

- **Codebook collision rate** – §II.A.3: OSQ-VAE reduces collisions to <0.5 % on every public catalog.
- **Catalog-size scaling** – §II.A.4: BARGE remains effective on a >300 k-item industrial catalog.
- **Domain shift** – §II.A.7: both BARGE and TIGER collapse under cross-catalog evaluation, as expected for catalog-specialized SID spaces.
- **Cold-start items** – §II.A.8: BARGE improves over TIGER on a 5 % cold-start split of Amazon Toys.

D5. Inconsistent candidates across channels

> Clarify how item IDs are mapped back when two channels produce inconsistent candidates.

Reply:

Each decoder first produces SID candidates, which are then mapped back to items through the corresponding SID-to-item index. If a SID maps to multiple items because of a collision, all mapped items receive the same probability contribution (internal order randomized). The candidate sets from different decoders are then merged by a **union** operation and truncated to the final top-k list.

Consequently the final recommendation list is defined at the **item level**; the inconsistent-candidate case described in the review does not arise. The tokenizer collision rate is also very low (see §II.A.3).

> We sincerely thank all reviewers for your thorough and diligent efforts in reviewing our work. Your insightful comments and constructive feedback have greatly strengthened the quality of this paper. We truly appreciate your time and expertise, and we wish you all the best!

Part II. Additional Experiments

II.A Common Experimental Evidence

II.A.1 Results on Amazon Toys

To address concerns about relying on previously reported results, we added Amazon Toys and reproduced the competing baselines ourselves.

Method	R@5	N@5	R@10	N@10
SASRec	0.0474	0.0321	0.0666	0.0382
TIGER	0.0499	0.0325	0.0769	0.0412
HSTU	0.0532	0.0390	0.0713	0.0448
ActionPiece	0.0467	0.0306	0.0735	0.0392
APAO-pointwise	0.0512	0.0339	0.0772	0.0423
Promise	0.0493	0.0321	0.0746	0.0402
RPG [1]	0.0579	0.0388	0.0863	0.0479
BARGE	**0.0622 ± 0.0018**	**0.0439 ± 0.0019**	**0.0886 ± 0.0012**	**0.0524 ± 0.0012**

BARGE consistently achieves the strongest results.
This experiment also directly addresses the concern that the reported improvements might depend on reusing metrics from previous papers.

[1] Hou et al., *Generating long semantic ids in parallel for recommendation*, KDD 2025.

II.A.2 Effectiveness and Efficiency of DPD

To isolate whether the gain of DPD comes merely from having additional decoders, we compare the following variants:

- **Module-free (top-20/top-40):** a faithful TIGER-style reproduction with the same codebook and no additional BARGE modules.

- **DPD → Ensemble:** replace OSQ-VAE with the original RQ-VAE, while keeping two independently trained decoders. This isolates the effect of simply adding two decoder towers.
- **Fixed random matrix:** replace the learned orthogonal transformation in OSQ-VAE with a fixed random matrix.
- **BARGE:** the complete model.

Amazon Beauty

Variant	R@5	N@5	R@10	N@10
Module-free (top20)	0.0450	0.0309	0.0696	0.0389
Module-free (top40)	0.0473	0.0317	0.0735	0.0402
DPD → Ensemble	0.0601	0.0412	0.0879	0.0502
Fixed random matrix	0.0591	0.0420	0.0845	0.0502
BARGE	**0.0654 ± 0.0006**	**0.0460 ± 0.0015**	**0.0927 ± 0.0004**	**0.0547 ± 0.0007**

Amazon Sports

Variant	R@5	N@5	R@10	N@10
Module-free (top20)	0.0257	0.0168	0.0405	0.0216
Module-free (top40)	0.0259	0.0169	0.0420	0.0221
DPD → Ensemble	0.0298	0.0199	0.0452	0.0248
Fixed random matrix	0.0309	0.0213	0.0472	0.0265
BARGE	**0.0369 ± 0.0009**	**0.0252 ± 0.0016**	**0.0544 ± 0.0007**	**0.0308 ± 0.0008**

These comparisons provide two useful controls. First, increasing the candidate beam size from 20 to 40 does not explain the full gain. Second, two decoders alone provide a substantial but clearly smaller improvement than BARGE. Replacing the learned orthogonal transformation with a fixed random matrix also reduces performance. Taken together, the results support our claim that

the improvement comes from the **joint design of multi-view representation, DPD, and the remaining BARGE modules**, rather than from decoder multiplicity alone.

Efficiency analysis

ICA and HPR are lightweight components. In our profiling, they contribute only a small fraction of end-to-end inference time. HPR computes its scores in a single forward pass; consequently, changing the top-N shortlist size has little observable effect on latency on the public datasets.

DPD does increase GPU memory consumption because two decoder towers, their KV-cache, and beam tensors must be maintained. This is an intentional memory/utilization trade-off: the additional GPU resources are exchanged for higher recommendation quality without introducing a new serial inference stage. The two view-specific decoders operate on the shared encoder memory and can execute side by side on the GPU.

II.A.3 Codebook Collision Rate

We further examine whether the multi-view tokenizer introduces excessive SID collisions. Let N denote the number of catalog items and U the number of distinct SIDs assigned by the tokenizer. We define the aggregate collision rate as $\mathrm{CollisionRate} = 1 - \frac{U}{N}$.

A value of 0% means that every catalog item receives a unique SID.

Dataset	# items N	Tokenizer	# unique SIDs U	Collision rate
Amazon Beauty	12,101	RQ-VAE	11,801	2.48%
Amazon Beauty	12,101	OSQ-VAE	12,096	**0.04%**
Amazon Sports	18,357	RQ-VAE	17,743	3.34%
Amazon Sports	18,357	OSQ-VAE	18,284	**0.40%**
Amazon Toys	11,924	RQ-VAE	11,660	2.21%
Amazon Toys	11,924	OSQ-VAE	11,886	**0.32%**

OSQ-VAE reduces the collision rate on all three datasets and keeps the rate below 0.5%. This is particularly relevant to DPD because each decoder operates on a complementary SID view; excessive collisions would otherwise make the view-to-item mapping ambiguous.

II.A.4 Catalog-Size Scaling and Industrial Results

The public Amazon datasets contain roughly 10^4 items. To examine whether BARGE depends on a small catalog, we additionally evaluate it on an industrial dataset containing:

- **>100M users**
- **>300k items**
- **>100M interactions/samples**

The scale is therefore substantially larger than the public benchmarks.

Method	H@5	H@10	H@20	H@50
GraphSAGE-based [2]	0.2932	0.3743	0.4650	0.5951
NANN [3]	0.4416	0.4946	0.5636	0.6760
OneRec	0.5459	0.6132	0.6729	0.7348
BARGE	**0.6015***	**0.6510***	**0.6967***	**0.7520***

$^*: p < 0.05.$

The results show that BARGE remains effective when moving from a roughly 10^4-item academic catalog to a catalog with more than 300k items and more than 100M interactions.

[2] Hamilton et al., *Inductive representation learning on large graphs*, NeurIPS 2017.

[3] Chen et al., *ANN search under neural similarity metric for large-scale recommendation*, CIKM 2022.

II.A.5 Industrial Online A/B Test

We also deploy BARGE in a full-scale online A/B test on the industrial recommendation platform described above, comparing it against the incumbent production baseline.

Metric	p-value	Significant?
Exposure conversion rate	1.08×10^{-5}	✓ ($p < 0.05$)
Platform total watch duration	0.04	✓ ($p < 0.05$)

Both core online metrics pass the significance threshold.

The statistical procedure uses two-group hypothesis testing. Regular metrics are evaluated with a T-test, while small-sample, long-tail, skewed, or extreme-value metrics use the rank-sum test (Mann--Whitney U). CUPED is used for variance reduction. For multiple metrics/comparisons, metrics are stratified by priority and corrected using BH-FDR, with significance declared at $q \leq 0.05$.

II.A.6 Inference Latency Breakdown

We profile inference on the Amazon Toys test set and aggregate the timing tags into the four major components of BARGE.

Module	Share of total inference time
Encoder (including ICA)	**1.84%**
DPD (per-view decoding + prefix filtering + view-to-item mapping)	**94.63%**
HPR (path embedding + dual-tower scoring + fusion)	**0.98%**
Other	≈ 2.55%

Two observations are important.

1. **HPR is lightweight at inference.** Its main operations are two small linear projections and a dot product, so the three HPR timing tags together account for only about 1% of total inference time.
2. **DPD is parallel multi-view autoregressive decoding rather than an additional serial stage.** Each view performs a standard autoregressive beam search over the shared encoder memory. There is no extra sequential pipeline stage between the two decoders. The main additional cost is GPU memory for the second decoder, KV-cache, and beam tensors. Thus, DPD trades memory/GPU utilization for recommendation quality while avoiding a proportional increase in serial latency.

For reproducibility, our public evaluation uses the same inference batch, beam width, and numerical precision across compared models. The exact hardware configuration may be adjusted according to GPU capacity; the reported comparisons are measured under the same configuration for each compared model.

II.A.7 Domain Shift

We additionally evaluate cross-domain transfer by taking checkpoints trained on Amazon Sports and directly evaluating them on Amazon Beauty without fine-tuning.

Both BARGE and TIGER produce essentially zero recommendation quality under this setting. This behavior is expected for SID-based generative recommendation: SIDs are constructed for a specific catalog, and the generator learns the SID distribution associated with that catalog. SIDs generated for another catalog are not guaranteed to share the same semantic code space.

Therefore, BARGE, like this family of SID-based generative recommenders, trades cross-domain reuse for a domain-specialized representation. The result also clarifies the intended scope of the proposed method: BARGE is designed to improve within-catalog generative retrieval rather than to provide a universal cross-domain SID space.

II.A.8 Cold-Start Item Experiment

Following the TIGER protocol, we construct a cold-start item set by deduplicating the last item of each user sequence and randomly sampling 5% of the resulting items as cold-start items. During training, these items are removed from both user histories and training targets. The tokenizer is also trained only on the remaining non-cold items.

After the tokenizer is frozen, it is used to assign SIDs to the held-out cold-start items. Evaluation is then performed on the complete test set. The codebook is fixed to `[256, 256, 256] + random suffix` for fair comparison.

On Amazon Toys, the split contains 374 cold-start items out of 7,475 (5%, seed=42) and 18,179 training samples.

Method	R@5	N@5	R@10	N@10
TIGER	0.0378	0.0244	0.0563	0.0303
BARGE	**0.0464**	**0.0325**	**0.0644**	**0.0383**

BARGE improves over TIGER on all four metrics in this cold-start setting, suggesting that the proposed multi-view representation and generation strategy can still benefit items whose interactions are withheld during generator training, provided that their content can be encoded by the frozen tokenizer.

II.A.9 ICA versus Simple Pooling

A key concern was whether ICA merely introduces a conventional item-level pooling bias. We therefore replace ICA with a simple mean-pooling operation over the same item-level sub-token features, keeping the rest of the model unchanged.

Variant	R@5	N@5	R@10	N@10
Mean-pooling	0.0599	0.0434	0.0838	0.0512
BARGE (ICA)	**0.0622**	**0.0439**	**0.0886**	**0.0524**

ICA outperforms simple mean pooling on all four metrics. More importantly, the conceptual distinction is that ICA does not "recover" a boundary that is literally unavailable. Item boundaries remain positionally recoverable from the fixed L-token layout. Instead, ICA explicitly constrains the encoder to form an item-level context representation and then adaptively injects that context back into the corresponding sub-token positions through a learned gate. The experiment above therefore tests whether this structured, content-dependent aggregation is useful beyond direct pooling.

Part III. Implementation Details

I.1 Overview and notation

Data flow. Given a user history $u = (v_1, \ldots, v_T)$, every item v_t is turned by a frozen OSQ-VAE into an $L \times H$ block of discrete codes. Flattened per item, the model consumes a sequence of length $T \cdot L \cdot H$ (interleaved per layer: `[l0_h0, l0_h1, l1_h0, l1_h1, ...]`). An encoder-decoder Transformer then autoregressively produces the $L \times H$ codes of the next item, one H-tuple per RQ layer.

Symbols used throughout.

Symbol	Meaning	Typical value
B	batch size	256
T	history length (items)	20
L	number of RQ layers	4
H	number of views (heads)	2
D	token embedding dim	128
D_z	RQ-VAE latent dim	32
D_{path}	HPR projection dim	128

Symbol	Meaning	Typical value
C_ℓ	codebook size at RQ layer ℓ	$[512, 256, 128, 64]$

Modules added by BARGE (all live on top of a TIGER-style backbone).

Name	Where it plugs in	Purpose
OSQ-VAE	Tokenizer	Turn one embedding into H complementary code streams.
`_compute_ica_context`	Encoder input, before position embeddings	Item-level context gating on the fused sub-token stream.
`n_heads` decoder towers	Per-view decoder	Independent per-view AR decoding sharing only encoder memory.
`HierarchicalPathReranker`	Per decoder tower, per layer $\ell \geq 1$	Path-level dual-tower InfoNCE reranker.
`_fuse_per_view_scores`	Post beam-search	Fusion across H views.

I.2 OSQ-VAE tokenizer

Implemented by `MvRqVae` and `MultiHeadQuantize` (referred to **OSQ-VAE**).

Shapes. Given $x \in \mathbb{R}^{B \times D_{\text{text}}}$:

- `encoder(x)` $\rightarrow z \in \mathbb{R}^{B \times D_z}$ ($D_z = 32$).
- optional learned rotation $z \leftarrow \text{rotation}(z)$ where `rotation` is an `nn.Linear(D_z, D_z, bias=False)` wrapped in `nn.utils.parametrizations.orthogonal(...)`, initialised to identity so training starts equivalent to no rotation.
- `torch.chunk(z, H, dim=-1)` $\rightarrow H$ tensors of shape $(B, D_z/H)$ = disjoint subspaces (no auxiliary orthogonality loss needed).

Per-layer quantization. Each RQ layer owns H independent codebooks `self.embeddings[h] = nn.Embedding(C_\ell, D_z / H)`. Given the current residual `res` (initially z), each layer runs:

```
for h in 0..H-1:
    dist_h    ← cosine(res_chunk_h, codebook_h)              # (B, C_ℓ)
    ids_h     ← argmin(dist_h, dim=-1)                        # (B,)
    emb_out_h ← quantize_forward(res_chunk_h, ids_h)          # (B, D_z/H)
        # forward mode: GUMBEL_SOFTMAX (default) | STE | ROTATION_TRICK
emb  ← concat(emb_out_0, ..., emb_out_{H-1}, dim=-1)          # (B, D_z)
ids  ← stack(ids_0, ..., ids_{H-1}, dim=1)                    # (B, H)
res  ← res - emb                                              # residual update
```

The Gumbel-softmax path is used during training. The decoder receives $\sum_\ell \hat{r}_\ell$ (with inverse rotation) and is trained with the standard MSE + commitment loss.

Output structure. After Stage-1 training, each item is stored as `sem_ids_structured ∈ ℤ^{B × L × H}` and `sem_ids ∈ ℤ^{B × L·H}`.

I.3 Item Context-Aware Attention (ICA)

Implemented in `_compute_ica_context`.

Placement. Inside `_prepare_encoder_input`, ICA is applied to the **already multi-view fused** sub-token stream, **before** adding position embeddings. It only exists on the encoder side.

Learnable parameters.

Name	Shape
`ica_query`	$(1, 1, D)$
`ica_attn`	`nn.MultiheadAttention(embed_dim=D, num_heads=max(1, D//32))`
`ica_norm`	`LayerNorm(D)`
`ica_proj`	`Linear(D, D) → GELU → Linear(D, D)`
`ica_gate`	`Linear(2·D, D) → Sigmoid`

Call chain. Input `seq_emb ∈ (B, T·L, D)` after multi-view fusion, `seq_mask ∈ (B, T·L)`:

```
# (1) group per item
item_tokens    ← seq_emb.view(B, T, L, D)
kv_tokens_flat ← item_tokens.reshape(B*T, L, D)
kv_mask_flat   ← seq_mask.view(B*T, L)          # True = valid

# (2) 1-token cross-attention against each item's L sub-tokens
q            ← ica_query.expand(B*T, 1, D)
ctx, _       ← ica_attn(q, kv_tokens_flat, kv_tokens_flat,
                        key_padding_mask=~kv_mask_flat)   # (B*T, 1, D)
ctx          ← ica_norm(ctx.squeeze(1))                    # (B*T, D)
# all-padded rows are set to 0 to prevent NaN propagation

# (3) project + broadcast back to every sub-token position
ctx_proj     ← ica_proj(ctx.view(B, T, D))                 # (B, T, D)
ctx_flat     ← ctx_proj.unsqueeze(2).expand(-1, -1, L, -1).reshape(B, T*L, D)

# (4) sigmoid-gated residual add
gate         ← ica_gate(concat([seq_emb, ctx_flat], dim=-1))  # (B, T·L, D)
ica_delta    ← gate * ctx_flat                                # returned
src          ← seq_emb + ica_delta                            # done in caller
```

Notes.

- Cost is $O(B \cdot T \cdot L \cdot D)$ (KV length is L, typically 4), strictly linear in sequence length.
- The gate is fed by both the original sub-token and the projected item-context, letting the model down-weight ICA when the item is already unambiguous.
- ICA has no dedicated loss; it is trained purely by the downstream NTP gradient.

I.4 Dual-Path Decoding with OR-fusion (DPD)

Two decoder towers (`self.dual_decoder=True` , `n_heads=H`):

Component	Storage	Shared across views?
Encoder	single	✓

Component	Storage	Shared across views?
`sparse_embedder_heads[h]` (per-view SID emb tables)	per view	✗
`start_token_embeddings[h]` (BOS)	per view	✗
`transformer_decoders[h]`	per view	✗
`sparse_output_projections[ℓ][h]` (D_attn → C_ℓ)	per view, per layer	✗
`path_rerankers[h]` (§I.5)	per view	✗
`reranker_path_embedders[h][ℓ]` (§I.5)	per view, per layer	✗

Both decoders cross-attend to the same encoder memory. Their self-attention only sees the SIDs of their own view (the training-time teacher-forcing stream is view `h`'s `L` codes). The two towers therefore operate on the two disjoint sub-vector streams produced in §I.2 without interfering during autoregressive decoding.

Per-view beam search. `generate_per_view_sem_id` runs H independent beam searches over the L RQ layers using `sparse_output_projections[ℓ][h]` for the logits at layer ℓ. Each view returns

```
sids_h  ∈ (B, K, L)   # per-view SID candidates
lps_h   ∈ (B, K)      # NTP log-probabilities of each beam
```

Per-view item scoring (loop in `generate_recommendations`, one per batch row):

- For each candidate SID `s = (c_1, ..., c_L)` from view h, look up the item collision set $\Phi_h(s) \subseteq \mathcal{V}$ via `self.item_ids_by_view[h]`.

Cross-view OR-fusion (`_fuse_per_view_scores`). All items that appear in any view's pool form a single union candidate set, and the

per-item scores from the views that contain it are combined by a configurable reduction. We use `lse` in the main experiments.

I.5 Hierarchical Path Reranker (HPR)

Implemented by `HierarchicalPathReranker`. One instance per view (`path_rerankers[h]`).

Parameters. For each RQ layer $\ell \in \{0, \dots, L-1\}$:

Name	Definition
`context_projs[ℓ]`	`Linear(D_attn, D_path)` (`D_path = 128`)
`path_projs[ℓ]`	`Linear(D, D_path)`
`log_temperatures[ℓ]`	`nn.Parameter(log(1/0.07))`, clamped to `[log(0.01), log(100)]`

Both projections are followed by `F.normalize(..., dim=-1)`, so the similarity is a cosine.

Path embedding tables. HPR uses **private** per-view, per-layer codeword tables `reranker_path_embedders[h][ℓ] = nn.Embedding(C_ℓ, D)`, decoupled from `sparse_embedder_heads`. This isolates the reranker's representation from the NTP output head's representation, which is what we found stable in practice.

Score for one layer. Given a beam candidate at layer ℓ with cumulative path embedding $p_\ell = \sum_{j=0}^{\ell} E_j^{(h)}[c_j]$:

```
ctx     ← normalize(context_projs[ℓ](h0))              # (B, D_path)
path    ← normalize(path_projs[ℓ](p_ℓ))                # (B, D_path)
score   ← (ctx * path).sum(-1) * temp[ℓ].clamp(0.01, 100)
```

where `h0` is the **BOS-position decoder hidden state** of view h.

Training loss. `compute_all_layers_loss` — for each view, the mean of per-layer symmetric InfoNCE across $\ell = 0, \dots, L-1$.

Per-layer loss (`compute_layer_loss_infonce`):

- Positives: diagonal of the in-batch cosine matrix `logits = ctx @ path.T * temp`.

- **Duplicate masking**: rows with identical cumulative path embedding (i.e. same SID prefix) have their off-diagonal entries set to -10^4.
- Optional **Method-B structured hard negatives** (`_build_hard_negatives`), controlled by `reranker_hard_neg_swap_current` and `reranker_hard_neg_swap_previous` :
  - *swap_current*: `GT_prefix[0..ℓ-1] + non-GT code at layer ℓ` .
  - *swap_previous*: full GT path with one earlier layer $k < \ell$ replaced by a non-GT code.
  - Hard negatives are appended only to the c2p direction's denominator so labels stay `arange(B)` ; the p2c direction stays symmetric in-batch. Warm-up is controlled by `reranker_hard_neg_warmup_epochs` .
- The per-layer loss is $(\mathcal{L}_{\text{c2p}} + \mathcal{L}_{\text{p2c}})/2$.

Inference-time fusion in beam search (excerpt from `generate_per_view_sem_id` , one view, one layer):

```
# expanded_log_probas: (B, pre_prune_k) NTP log-probs of expanded candidates
# new_path_embs       : (B, pre_prune_k, D) cumulative path embs via HPR-private tables
h_ctx_exp    ← bos_hidden_h.unsqueeze(1).expand(-1, pre_prune_k, -1)
reranker_lp  ← log_softmax(path_rerankers[view_h].score(
                     h_ctx_exp, new_path_embs, layer_idx), dim=-1)
fused_scores ← expanded_log_probas + λ_ℓ * reranker_lp
# then top-K pruning by fused_scores; carry-forward log-prob is NTP only
```

I.6 Training objective and two-stage pipeline

Stage 1 — tokenizer. Train the OSQ-VAE tokenizer with

```
loss = reconstruction_loss + quan_loss_weight * quantize_loss
```

`quantize_loss` is the standard VQ commitment loss summed over all $L \cdot H$ code positions. Then freeze the tokenizer and pre-compute SIDs for the whole item catalog.

Stage 2 — generator. The generator loss (in `Barge.forward`) is

```
loss = Σ_h Σ_ℓ CE(logits_{h,ℓ},   gt_c_{h,ℓ}, label_smoothing=0.1)
       + reranker_loss_weight * (1/H) Σ_h  HPR_loss_h
```

I.7 Inference procedure

`generate_recommendations` runs the following pipeline per batch. Below, `H` is the number of views, `K` the number of recommendations returned, `K_wide` a per-layer wide beam width used to build the union pool.

```
def recommend(batch, K):
    # (1) shared encoder pass (ICA lives inside _prepare_encoder_input)
    memory ← encoder(_prepare_encoder_input(batch))

    # (2) per-view beam search over L RQ layers
    per_view = []
    for h in 0..H-1:
        beam        ← BOS_h
        beam_lp     ← 0
        beam_path   ← 0                                    # cumulative path emb
        h0          ← decoder_h(beam).bos_hidden           # (B, D_attn)
        for ℓ in 0..L-1:
            logits          ← output_head[ℓ][h](decoder_h(beam, memory)[:, ℓ])
            log_p           ← log_softmax(logits, dim=-1)
            expand          ← beam × top_{pre_prune_k}(log_p)
            new_path_emb    ← beam_path + reranker_path_embedders[h][ℓ](new_code)
            if HPR active on ℓ:
                r_lp        ← log_softmax(path_rerankers[h].score(h0, new_path_emb, ℓ))
                fused       ← expand.log_p + λ_ℓ * r_lp
            else:
                fused       ← expand.log_p
            beam            ← top-K_ℓ prefixes ranked by fused
            beam_lp         ← NTP log-prob carried forward (not fused)
            beam_path       ← gather path embs matching pruned beam
        per_view.append((beam.sids, beam_lp))              # (B, K_wide, L), (B, K_wide)

    # (3) view -> item scoring (per view, per batch row)
    per_view_scores = [{} for _ in range(H)]
    for h, (sids_h, lps_h) in enumerate(per_view):
        for cand, lp in zip(sids_h, lps_h):
            items = item_ids_by_view[h][tuple(cand)]       # collision set
            if not items:  continue
            delta = lp - log(len(items))
            for v in items:
                per_view_scores[h][v] = lse_merge(per_view_scores[h].get(v), delta)

    # (4) OR-fusion across views (mode ∈ {lse, max, mean, rrf})
    fused = _fuse_per_view_scores(per_view_scores, mode=self.or_fusion_mode)

    # (5) sort and return top-K
    return top_K_by_score(fused, K)
```

Points worth stressing:

- Steps (2) and (3) run one view at a time and the results are only combined in (4); no cross-view state exists inside beam search.
- Within-view aggregation (step 3) and cross-view aggregation (step 4, `lse` mode) share the same LSE reduction, so the intra- and inter-view scales are consistent.

I.8 Hyper-parameters used in the paper

Tokenizer (Stage 1).

Hyper-parameter	Value
Item text embedding dim D_{text}	768
Encoder MLP hidden dims	$[512, 256, 128]$
RQ latent dim D_z	32
Number of RQ layers L	4
Codebook sizes $(C_1, \ldots, C_L)$	$[512, 256, 128, 64]$
Number of views H	2
Commitment weight β	0.25
Gumbel-softmax temperature	0.2
Learned Householder rotation R	enabled
Optimizer	AdamW, lr $1\text{e-}3$, wd $1\text{e-}3$
Iterations	$20{,}000$
Batch size	1024

Generator (Stage 2).

Hyper-parameter	Value
SID embedding dim	128
Attention hidden dim D_{attn}	512

Hyper-parameter	Value
Heads per Transformer block	4
Encoder layers	2
Decoder layers per view	2 (each of the H decoders)
Dropout	0.2
Max sequence length	256
Softmax temperature	1.0
Beam width $(K_1, \ldots, K_L)$	$[20, 20, 20, 20]$
Optimizer	AdamW, lr $1\text{e-}3$, wd 0.01
Batch size	256
Epochs	up to 200 (early stop on NDCG@20)
Label smoothing	0.1

BARGE-specific knobs.

Hyper-parameter	Value
ICA MHA heads	$D/32 = 4$
OR-fusion mode	`lse` (log-sum-exp)
HPR inference weight λ_ℓ	$[0.25, 0.25, 0.25, 0.25]$
HPR shortlist size N_ℓ	$[400, 400, 400, 400]$
HPR active layers $\mathcal{L}_{\text{on}}$	$\{0, 1, 2, 3\}$